

Paradigmas de Programação — Aula 08:

Concorrência, gorrotinas e canais

Exemplo: cotação em vários fornecedores

O problema

Fornecedor	Preço	Latência
Loja A	R\$ 129,90	100 ms
Loja B	R\$ 99,90	250 ms
Loja C	R\$ 114,50	180 ms

- O serviço precisa do **menor preço** entre os três
- Em sequência: $100 + 250 + 180 = \mathbf{530\ ms}$
- As consultas são **independentes** e passam o tempo **esperando**: o caso em que concorrência rende
- Latência simulada com `time.Sleep` ; num sistema real, uma chamada HTTP

Uma gorrotina por consulta, um canal de respostas

```
func consultar(f Fornecedor, respostas chan<- Fornecedor) {  
    time.Sleep(f.Latencia)  
    respostas <- f  
}  
  
respostas := make(chan Fornecedor, len(fornecedores))  
for _, f := range fornecedores {  
    go consultar(f, respostas)  
}
```

- *Buffer* do tamanho da lista: quem responder depois do prazo envia e termina, sem ficar bloqueado

O prazo com **select**

```
limite := time.After(prazo)           // criado uma vez, antes do laço
espera:
for recebidas < len(fornecedores) {
    select {
    case f := <-respostas:
        recebidas++                     // compara com o melhor preço até aqui
    case <-limite:
        break espera                   // sai do for, não só do select
    }
}
```

A medição

Prazo	Respostas, em ordem de chegada	Melhor	Tempo total
500 ms	A (100), C (180), B (250)	Loja B, R\$ 99,90	250 ms
200 ms	A (100), C (180); B descartada	Loja C, R\$ 114,50	200 ms

- As respostas chegam em **ordem de latência**, não na ordem da lista
- Prazo folgado: o tempo é o da consulta **mais lenta**, não a soma — as esperas se sobrepõem, sem precisar de três núcleos
- Prazo curto: o tempo é o **prazo**, e a mais barata fica de fora — previsibilidade trocada por qualidade da resposta

Os dois modelos lado a lado

Threads em Java	Gorrotinas e canais em Go
três threads	três gorrotinas
estrutura compartilhada com as respostas	canal de respostas
trava sobre a estrutura	só <code>cotar</code> lê e compara os preços
condição de corrida possível	nenhum dado compartilhado